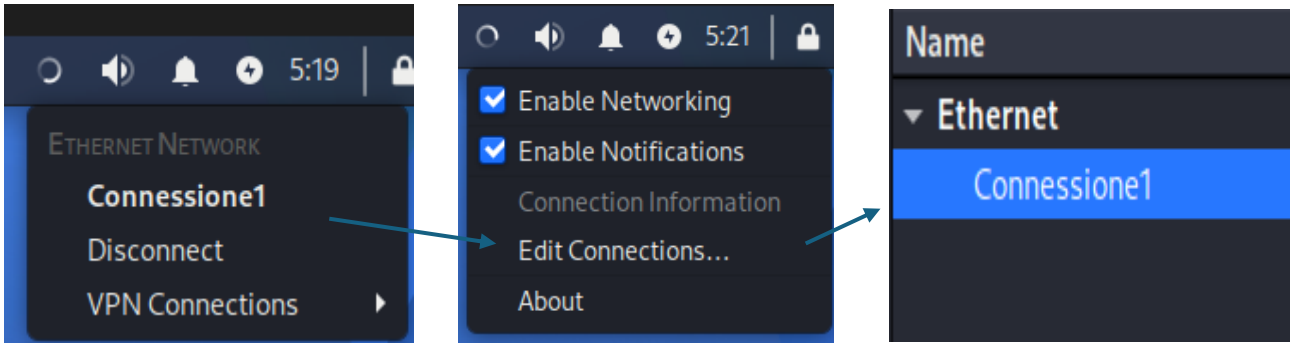
Analisi Architettura Client Server

Traccia: *Simulare, in ambiente di laboratorio virtuale, un'architettura client server in cui un client con indirizzo* ***192.168.32.101(Windows)*** *richiede tramite web browser una risorsa all'hostname* ***epicode.internal*** *che risponde all'indirizzo* ***192.168.32.100(Kali).***

Si intercetti poi la comunicazione con ***Wireshark****, evidenziando i* ***MAC Andress*** *di sorgente e destinazione ed il contenuto della richiesta* ***HTTPS****. Ripetere l'esercizio, sostituendo il server* ***HTTPS****, con un server* ***HTTP****. Si intercetti nuovamente il traffico, evidenziando le eventuali differenze tra il traffico appena catturato in* ***HTTP*** *ed il traffico precedente in* ***HTTPS****. Spiegare, motivandole, le principali differenze se presenti.*

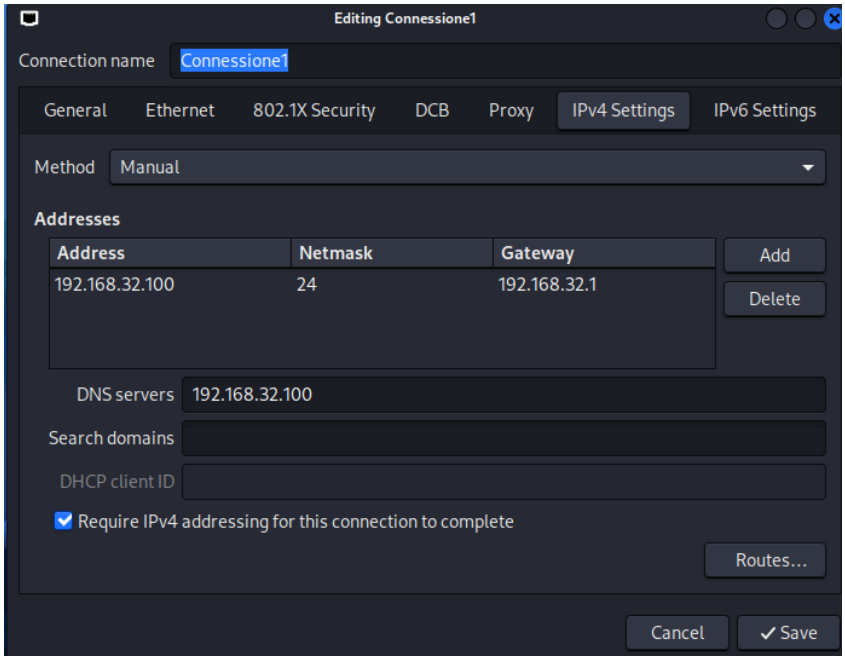
Configurazione delle Macchine Virtuali: Kali Linux

Per Configurare la nostra Kali Linux prima di tutto la avviamo da ***VirtualBox,*** *inseriamo le credenziali di accesso predefinite****("Kali"),*** *successivamente andiamo a modificare la rete ethernet precedentemente configurata ovvero* ***"Connessione1",*** *facendo tasto destro sull'icona ethernet e facendo "****edit connections">"connessione1">"edit":***



Dopo aver fatto ciò andremo su ***"IPv4 Settings"*** *e modificheremo* ***l'indirizzo IP*** *con il "****192.168.32.100****", il* ***Gateway*** *con l'indirizzo* ***"192.168.32.1"*** *e il* ***DNS*** *con l'indirizzo* ***"192.168.32.100".***

Successivamente clicchiamo su" ***Save"*** *per salvare la configurazione e se vogliamo possiamo provare nel terminale ad inserire la riga* ***"ip a"*** *per la verifica della connessione.*

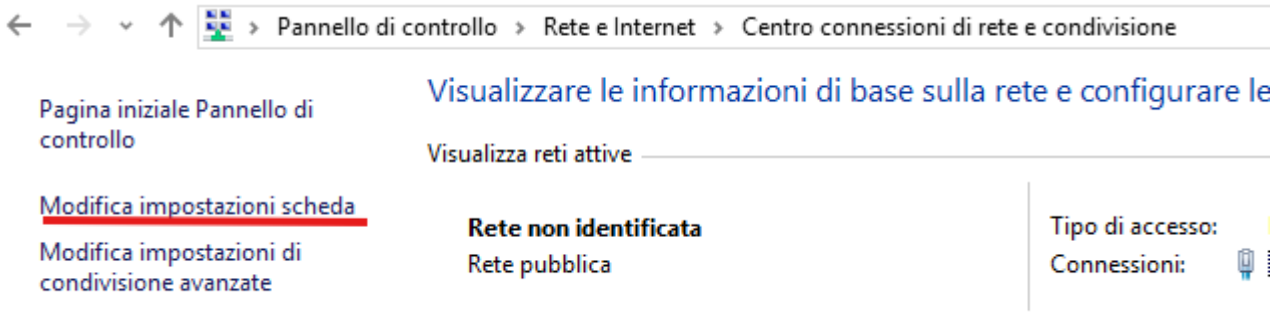


Configurazione Delle Macchine Virtuali: Windows

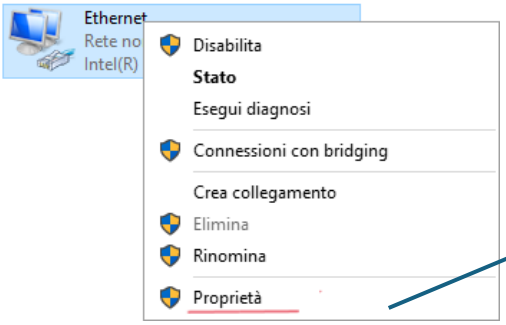
*Proprio come fatto per Kali, dobbiamo modificare l'indirizzo **IP, il Gateway e il DNS** della macchina **Windows** per permettere la corretta comunicazione tra le **VM**. Poiché la macchina **Windows** agisce come client che richiede una risorsa **all'hostname epicode.internal** (ospitato sulla macchina Kali), è fondamentale impostare l'indirizzo IP di Kali come Server DNS. In questo modo, le richieste di risoluzione del nome verranno intercettate da Kali, permettendo il corretto instradamento del traffico verso i servizi simulati.*

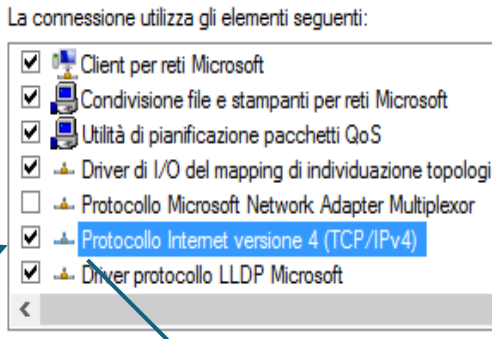
Configurazione Indirizzi:

Per la configurazione della Macchina Windows come per Kali andremo ad accendere da VirtualBox la VM. Successivamente digiteremo nella barra di ricerca "Pannello di Controllo">Centro di Connessioni di rete e condivisione">Modifica impostazioni Scheda.

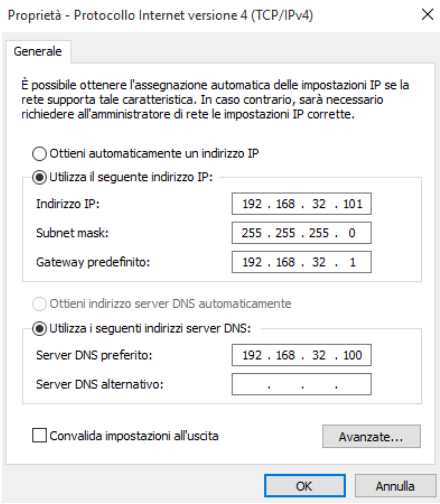


Si aprirà una finestra con una porta ethernet, facciamo tasto destro>"proprietà">"Protocollo Internet versione 4(TCP/IP)":





Cliccando su protocollo Internet si aprirà una scheda per la modifica degli indirizzi:



*Da questa scheda modifichiamo **L'indirizzo IP** inserendo l'indirizzo **192.168.32.101**, L'indirizzo del **Gateway Predefinito** inserendo l'indirizzo **192.168.32.1** e l'indirizzo del **Server DNS preferito** con l'indirizzo **192.168.32.100** ovvero l'indirizzo della macchina **Kali**. In questo modo **Windows** quando dovrà chiedere la risoluzione dei nomi saprà di doverli chiedere all'indirizzo **192.168.32.100**, quindi vedrà come **Server DNS** la macchina **Kali**.*

Configurazione Servizi: HTTPS

*Ritornando Su Kali proseguiamo con la configurazione del servizio HTTPS attraverso "**inetsim**" ovvero un emulatore di vari servizi. Per fare ciò ci avvalleremo del terminale in cui modificheremo il file di inetsim per avviare solamente il servizio HTTPS.*

*Digitiamo il comando **"sudo nano /etc/inetsim/inetsim.conf" per modificare il file:***

```
kali@kali: ~
Session  Actions  Edit  View  Help
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ sudo nano /etc/inetsim/inetsim.conf
```

Facciamo Invio, successivamente andiamo a cercare nel file la voce" #start service https" e leviamo il "#" così da non leggerlo più come commento ma comando:

```
kali@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.7.1            /etc/inetsim/inetsim.conf *
# start_service
#
# The services to start
#
# Syntax: start_service <service name>
#
# Default: none
#
# Available service names are:
# dns, http, smtp, pop3, tftp, ftp, ntp, time_tcp,
# time_udp, daytime_tcp, daytime_udp, echo_tcp,
# echo_udp, discard_tcp, discard_udp, quotd_tcp,
# quotd_udp, chargen_tcp, chargen_udp, finger,
# ident, syslog, dummy_tcp, dummy_udp, smtps, pop3s,
# ftps, irc, https
#
#start_service dns
#start_service http
start_service https
#start_service smtp
#start_service smtps
#start_service pop3
#start_service pop3s
#start_service ftp

^G Help      ^O Write Out  ^F Where Is   ^K Cut      ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste    ^J Justify
```

*Inoltre, dobbiamo andare a modificare un'altra parte del file, perché quando avviamo **inetsim** invece di ascoltare **l'indirizzo 192.168.32.100** ascolta di default l'indirizzo di **loopback** ovvero il **127.0.0.1**, per risolvere questo problema andiamo a modificare la parte "service_bind_address" nel file e inseriamo 0.0.0.0(ascolta tutte le interfacce) levando*

ovviamente il commento "#". In questo modo ascolterà le richieste di tutte le interfacce disponibili senza escluderne nessuna.

Il file risulterà in questa maniera:

```
##########################################
# service_bind_address
#
# IP address to bind services to
#
# Syntax: service_bind_address 192.168.32.100

# Default: 0.0.0.0
#
service_bind_address 0.0.0.0
```

Dopo aver fatto, salviamo il file con la combinazione ctrl+x + Y + INVIO.

Configurazione Servizi: DNS

*Per la configurazione del servizio **DNS** inetsim non può essere utilizzato dato che il servizio **DNS** non è funzionante. Per risolvere il problema ho optato l'utilizzo di un altro Fake DNS Server ovvero "**dnschef**" che si occupa esattamente di richieste **DNS**. Per configurare correttamente il servizio **DNS** tramite **dnschef**, utilizzeremo il comando: **"sudo dnschef --interface 192.168.32.100 --fakeip 192.168.32.100 --fakedomains epicode.internal"**. Con questa istruzione stiamo istruendo il **server DNS** a intercettare le richieste per il dominio **epicode.internal** e a rispondere al client (**Windows**) fornendo **l'indirizzo IP** della nostra macchina **Kali** (**192.168.32.100**). In questo modo, ogni tentativo di connessione al dominio verrà reindirizzato verso i servizi simulati sulla nostra macchina.*

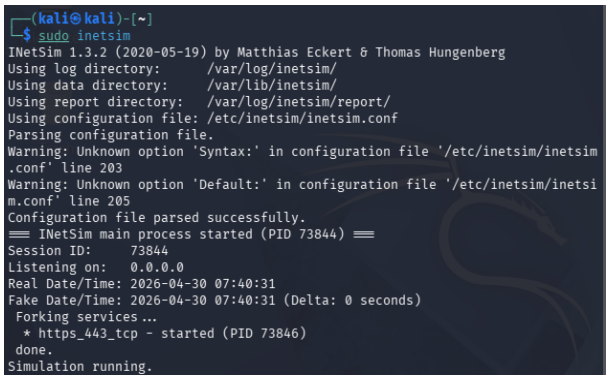
```
kali@kali: ~
Session  Actions  Edit  View  Help
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ sudo dnschef -i 192.162.32.100 --fakeip 192.168.32.100 --fakedomains epic
ode.internal
```

Dopo aver fatto INVIO vedremo il servizio in azione:

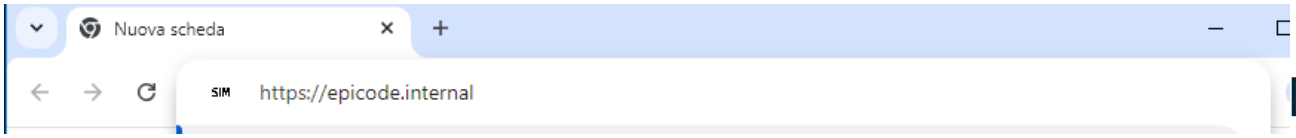
```
Session  Actions  Edit  View  Help
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ sudo dnschef -i 192.162.32.100 --fakeip 192.168.32.100 --fakedomains epic
ode.internal
[sudo] password for kali:
/usr/bin/dnschef:453: SyntaxWarning: invalid escape sequence '\/'
  header += "        / _` | '_ \/ __|/ __| '_ \ / _ \  _|\n"
/usr/bin/dnschef:454: SyntaxWarning: invalid escape sequence '\_'
  header += "        | (_| | | | \__ \ (__| | | |  __/ |  \n"
/usr/bin/dnschef:455: SyntaxWarning: invalid escape sequence '\_'
  header += "        \__,_|_| |_|___/\___|_| |_|\___|_|  \n"
                                                  __
          | | version 0.4   | |        / _|
       __| |_ __   ___  ___| |__   ___| |_
      / _` | '_ \ / __|/ __| '_ \ / _ \  _|
     | (_| | | | \__ \ (__| | | |  __/ |
      \__,_|_| |_|___/\___|_| |_|\___|_|
                    iphelix@thesprawl.org

(07:26:26) [*] DNSChef started on interface: 192.162.32.100
(07:26:26) [*] Using the following nameservers: 8.8.8.8
(07:26:26) [*] Cooking A replies to point to 192.168.32.100 matching: epicode
.internal
```

Comunicazione Client Server -HTTPS:

*Eseguiamo adesso la comunicazione tra **Client** e **Server** Avviando su **Kali inetsim** per il servizio **HTTPS** e **dnschef** Per il servizio **DNS**:*

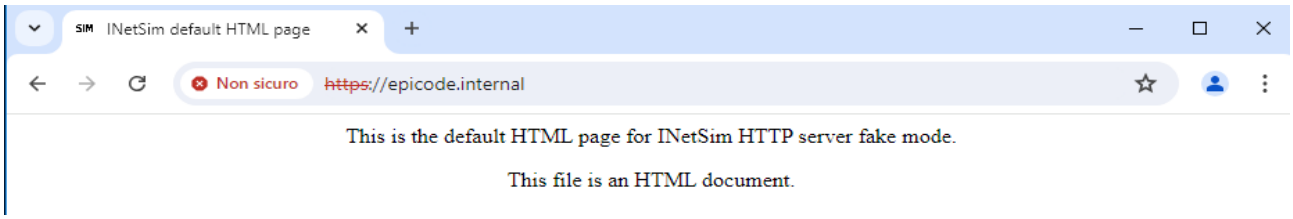


Mentre su windows andiamo su Google Crome e proviamo ad inserire il dominio "https://epicode.internal" e verifichiamo il funzionamento:



*Quando utilizziamo il protocollo **HTTPS windows** pretende che il **server**, in questo caso **inetsim** che lo emula, dimostri chi è tramite un **certificato SSL/TLS**. **Inetsim** allora genera un certificato falso ma **Windows** lo capisce e per questo ci avverte che la connessione non è sicura e se vogliamo avanzare o meno:*

*Ovviamente essendo una simulazione controllata facciamo avanza, e come possiamo vedere la comunicazione è perfettamente funzionante, permettendoci di caricare il fake file **HTML** creato da **inetsim**:*

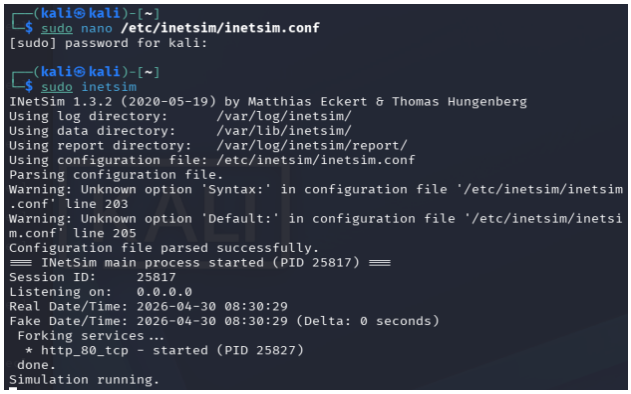


Comunicazione Client Server -HTTP:

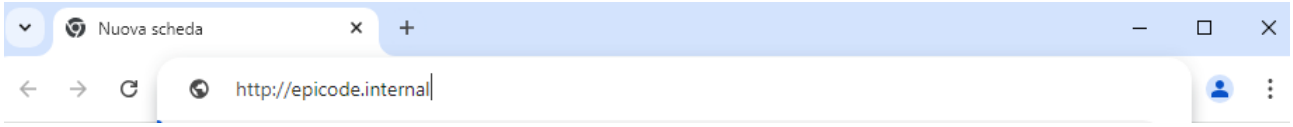
*Facciamo il medesimo test solamente che stavolta implementeremo il servizio **HTTP** per poi analizzare le differenze su **Wireshark** tra il **HTTPS** e **HTTP**, quindi l'unica modifica da apportare sarà su **inetsim** dove modificheremo nuovamente il file, spegneremo il servizio **HTTPS** e invece attiveremo il servizio **HTTP**:*

```
#
#start_service dns
start_service http
#start_service https
```

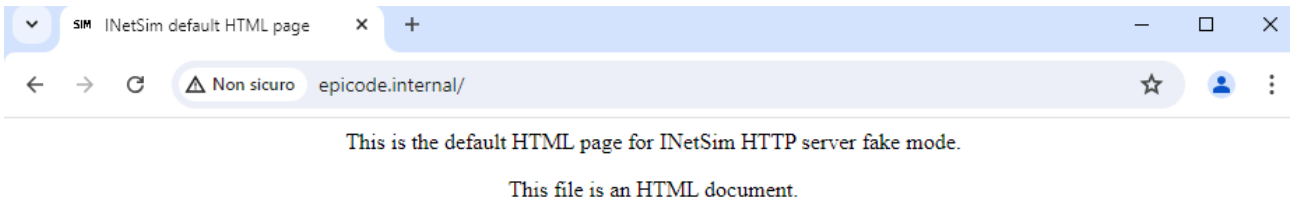
Fatto ciò, salviamo il file con la combinazione **ctrl+x + Y + INVIO** e facciamo partire sia inetsim che **dnschef**:



*Mentre su **Windows** andiamo su **Google Crome** inserendo stavolta il dominio "http://epicode.internal" e ne verifichiamo il funzionamento:*



Premiamo invio e come possiamo vedere *la comunicazione è perfettamente funzionante, permettendoci di caricare il fake file **HTML** creato da **inetsim**:*

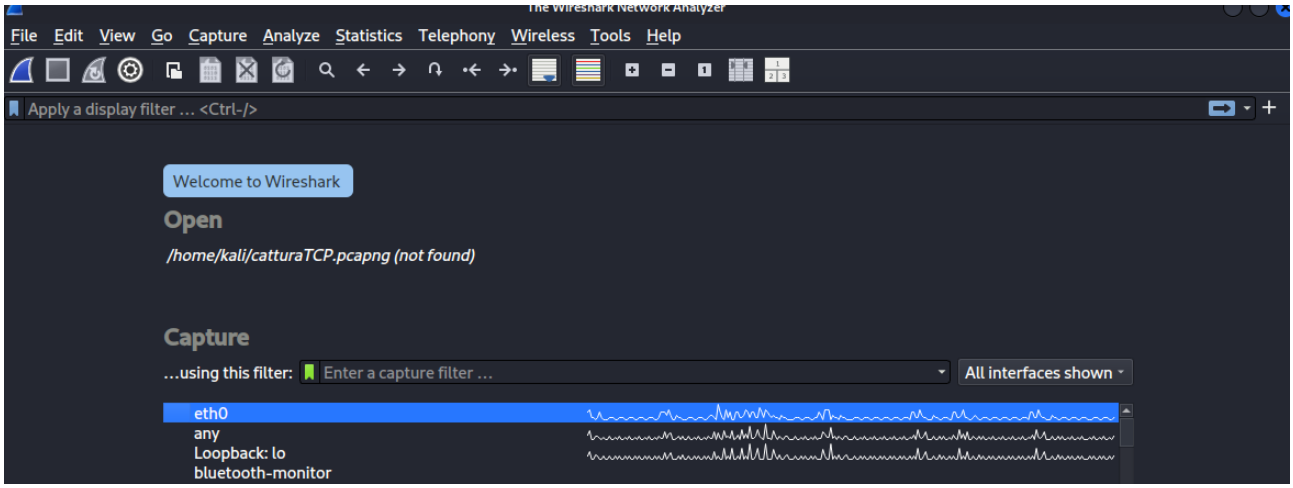


Analisi dei servizi HTTPS e HTTP tramite Wireshark:

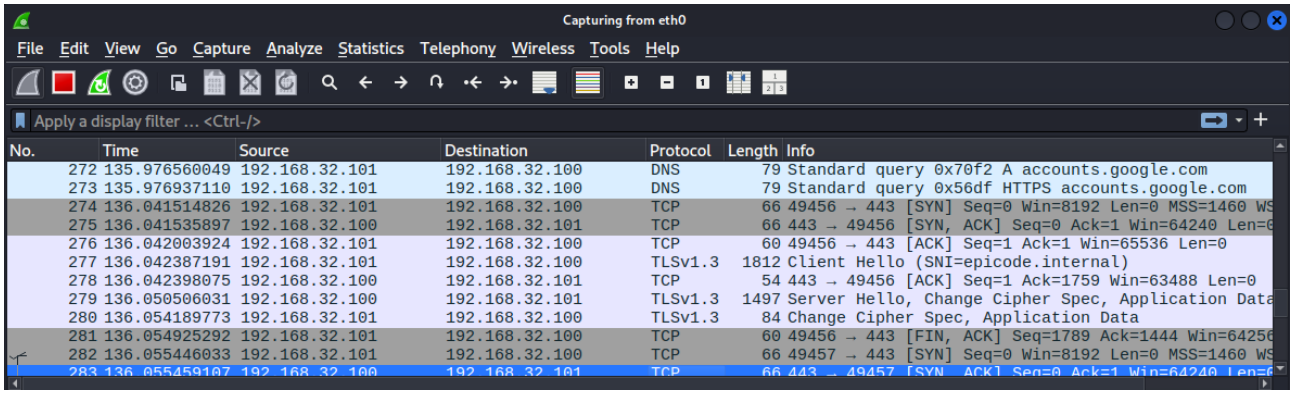
Analisi HTTPS: Durante la comunicazione tramite protocollo **HTTPS** avviamo **Wireshark** su Kali utilizzando il comando *"sudo wireshark"*:

```
┌──(kali㉿kali)-[~]
└─$ sudo wireshark
```

Facciamo invio avviando **Wireshark**:



Una volta avviata l'interfaccia grafica scegliamo l'interfaccia di rete **eth0** per avviare la cattura dei pacchetti. Durante la comunicazione tra **Windows** e **Kali**, filtreremo il traffico per isolare i pacchetti relativi al protocollo **HTTPS** (porta 443), analizzando così lo scambio di dati tra client e server simulato:



No.	Time	Source	Destination	Protocol	Length	Info
272	135.976560049	192.168.32.101	192.168.32.100	DNS	79	Standard query 0x70f2 A accounts.google.com
273	135.976937110	192.168.32.101	192.168.32.100	DNS	79	Standard query 0x56df HTTPS accounts.google.com
274	136.041514826	192.168.32.101	192.168.32.100	TCP	66	49456 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS
275	136.041535897	192.168.32.100	192.168.32.101	TCP	66	443 → 49456 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
276	136.042003924	192.168.32.101	192.168.32.100	TCP	60	49456 → 443 [ACK] Seq=1 Ack=1 Win=65536 Len=0
277	136.042387191	192.168.32.101	192.168.32.100	TLSv1.3	1812	Client Hello (SNI=epicode.internal)
278	136.042398075	192.168.32.100	192.168.32.101	TCP	54	443 → 49456 [ACK] Seq=1 Ack=1759 Win=63488 Len=0
279	136.050506031	192.168.32.100	192.168.32.101	TLSv1.3	1497	Server Hello, Change Cipher Spec, Application Data
280	136.054189773	192.168.32.101	192.168.32.100	TLSv1.3	84	Change Cipher Spec, Application Data
281	136.054925292	192.168.32.101	192.168.32.100	TCP	60	49456 → 443 [FIN, ACK] Seq=1789 Ack=1444 Win=64256
282	136.055446033	192.168.32.101	192.168.32.100	TCP	66	49457 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS
283	136.055459107	192.168.32.100	192.168.32.101	TCP	66	443 → 49457 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0

Avviando la ricerca su Windows e tenendo inetsim e dnschef accesi analizziamo i pacchetti che vengono scambiati: Qui il client (ovvero l'indirizzo 192.168.32.101) deve capire dove si trova il server.

- **Pacchetto 272:** Il client invia una **query** una richiesta per **accounts.google.com.** Sta chiedendo al server **DNS** (192.168.32.100) **l'indirizzo IP** del sito.
- **Pacchetto 273:** Viene inviata una **query** specifica per **HTTPS**. È una funzione moderna dei browser per velocizzare la connessione verificando subito se il sito utilizza protocolli sicuri o altre configurazioni.

- **Pacchetto 274-276 (TCP 3-Way Handshake)** prima di poter iniziare la comunicazione bisogna stabilire la connessione:

    1. **274[SYN]** sincronizzazione con la porta 443(HTTPS) del client
    2. **275[SYN-ACK]** il server risponde con un ACK ovvero come se dicesse "RICEVUTO" e "SINCRONIZZIAMOCI" con il SYN da parte sua.
    3. **276[ACK]** il client risponde con un ACK ovvero "RICEVUTO".

- **Pacchetti 277-280(TLS 1.3 Handshake)** qui viene utilizzato il TLS per la crittografia in particolare TLS 1.3 una versione più veloce e sicura:

    1. **277 Client Hello:** il client invia al server le sue "credenziali" inoltre utilizza lo SNI (Server Name Indicator) per dire al server quale "certificato" mostrare (in questo caso epicode.internal)
    2. **278[ACK]:** altra ACK di conferma.
    3. **279 Server Hello:** Il server sceglie il miglior algoritmo, invia il suo certificato e le chiavi per attuare la cifratura.
    4. **280 Change Cipher Spec:** Il client e Server confermano di usare solo la chiave simmetrica appena generata per comunicare.
- Pacchetti 281-283:
    1. **281[FIN, ACK]:** il client decide di chiudere la connessione il flag FIN(Finish) indica la normale chiusura.
    2. **282-283[SYN]**il client ricomincia subito una nuova connessione.

Analisi interna dei dati del Pacchetto 277: cliccando sul pacchetto 277 e andando in Ethernet II potremo visualizzare il Source e Destination MAC:

```
▸ Frame 277: Packet, 1812 bytes on wire (14496 bits), 1812
▾ Ethernet II, Src: PCSSystemtec_92:4e:9f (08:00:27:92:4e:9
  ▸ Destination: PCSSystemtec_8a:35:d2 (08:00:27:8a:35:d2)
  ▸ Source: PCSSystemtec_92:4e:9f (08:00:27:92:4e:9f)
    Type: IPv4 (0x0800)
    [Stream index: 7]
▸ Internet Protocol Version 4, Src: 192.168.32.101, Dst: 19
▸ Transmission Control Protocol, Src Port: 49456, Dst Port:
▸ Transport Layer Security
```

Se clicchiamo su Internet Protocol invece vedremo l'IP mittente e destinatario:

```
▸ Frame 277: Packet, 1812 bytes on wire (14496 bits), 1812 bytes captured (14496 bi
▸ Ethernet II, Src: PCSSystemtec_92:4e:9f (08:00:27:92:4e:9f), Dst: PCSSystemtec_8a
▾ Internet Protocol Version 4, Src: 192.168.32.101, Dst: 192.168.32.100
    0100 .... = Version: 4
```

come possiamo vedere il mittente è il Client Windows e il destinatario il Server (Kali).

Se invece clicchiamo su Transport vedremo le porte di mittente e destinatario:

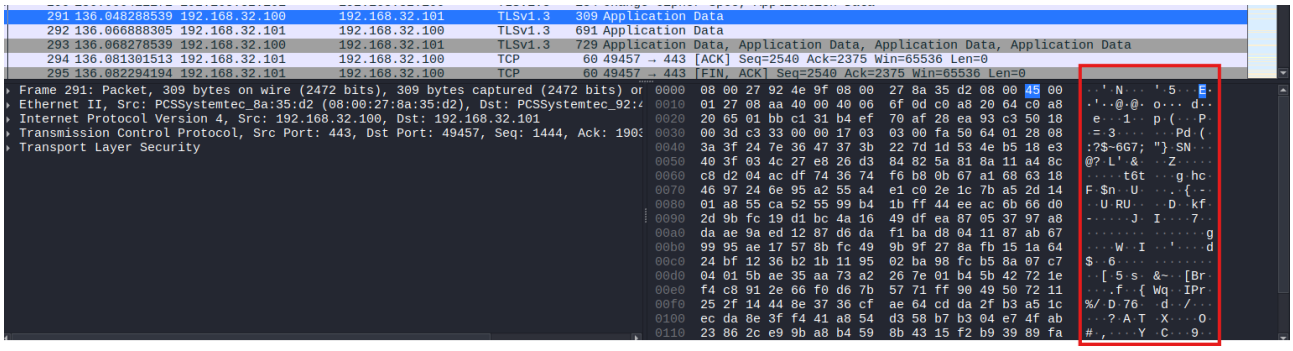
```
▸ Internet Protocol Version 4, Src: 192.168.32.101, Dst: 192.168.32.100
▾ Transmission Control Protocol, Src Port: 49456, Dst Port: 443, Seq: 1, Ack: 1, Le
    Source Port: 49456
    Destination Port: 443
```

Come possiamo vedere viene mostrata la porta randomica del mittente n 49456 e la porta di destinazione la porta 443 ovvero HTTPS.

Se invece Analizziamo il Transport Layer Security (application layer) possiamo vedere l'Handshake Protocol: Client Hello: Indica l'inizio della negoziazione crittografica. Inoltre, anche il Server Name Indicator (SNI): epicode.internal. serve al server per capire quale certificato SSL mostrare.

Livello OSI	Nome Livello	Informazione Chiave	Valore nel Frame 277
Layer 2	Data Link	MAC Source	`08:00:27:92:4e:9f`
Layer 2	Data Link	MAC Destination	`08:00:27:8a:35:d2`
Layer 3	Network	IP Source	`192.168.32.101`
Layer 3	Network	IP Destination	`192.168.32.100`
Layer 4	Transport	Port Dest (HTTPS)	`443`
Layer 7	Application	Estensione SNI	`epicode.internal`

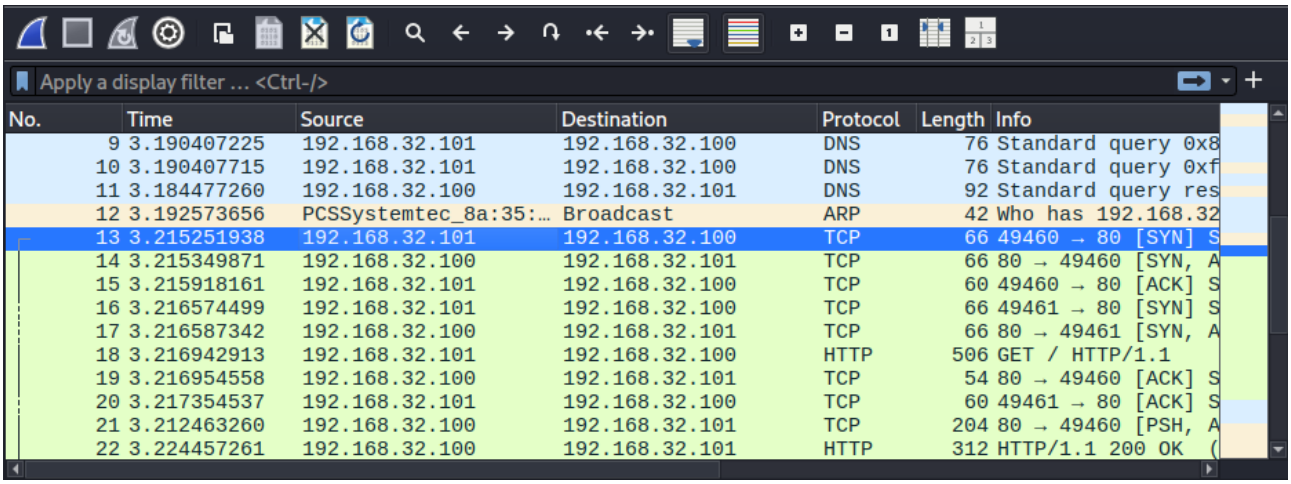
Analisi Pacchetto Application Data: Se andiamo ad analizzare un pacchetto dati ovvero il pacchetto contenente dati veri come, per esempio, il codice di una pagina HTML o una richiesta JSON vedremo che i dati saranno cifrati da TLS e illeggibili per noi:



Inoltre, analizzando la cattura dei pacchetti precedentemente mostrata possiamo notare che come il tempo tra il DNS e inizio del TCP sia quasi nullo mentre l'Handshake TLS occupa la maggior parte dei millisecondi. Questo ci fa capire perché HTTPS è leggermente più pesante rispetto a HTTP (ma ovviamente migliore e necessario per la sicurezza).

Analisi HTTP:

Vediamo invece la cattura dei pacchetti con l'utilizzo del protocollo HTTP invece di HTTPS, questo sempre tramite wireshark , avviando la richiesta da Google sul client(Windows) e tenendo sempre accesi inetsim e dnschef sul Server(Kali):



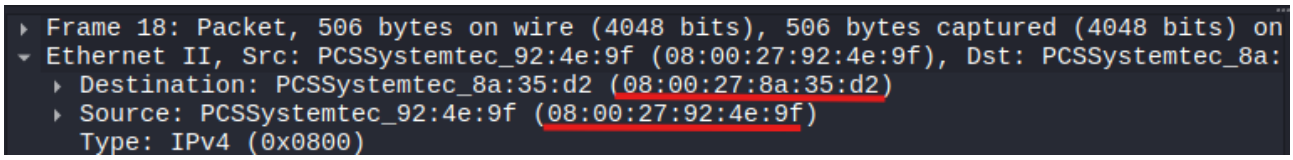
No.	Time	Source	Destination	Protocol	Length	Info
9	3.190407225	192.168.32.101	192.168.32.100	DNS	76	Standard query 0x8
10	3.190407715	192.168.32.101	192.168.32.100	DNS	76	Standard query 0xf
11	3.184477260	192.168.32.100	192.168.32.101	DNS	92	Standard query res
12	3.192573656	PCSSystemtec_8a:35:…	Broadcast	ARP	42	Who has 192.168.32
13	3.215251938	192.168.32.101	192.168.32.100	TCP	66	49460 → 80 [SYN] S
14	3.215349871	192.168.32.100	192.168.32.101	TCP	66	80 → 49460 [SYN, A
15	3.215918161	192.168.32.101	192.168.32.100	TCP	60	49460 → 80 [ACK] S
16	3.216574499	192.168.32.101	192.168.32.100	TCP	66	49461 → 80 [SYN] S
17	3.216587342	192.168.32.100	192.168.32.101	TCP	66	80 → 49461 [SYN, A
18	3.216942913	192.168.32.101	192.168.32.100	HTTP	506	GET / HTTP/1.1
19	3.216954558	192.168.32.100	192.168.32.101	TCP	54	80 → 49460 [ACK] S
20	3.217354537	192.168.32.101	192.168.32.100	TCP	60	49461 → 80 [ACK] S
21	3.212463260	192.168.32.100	192.168.32.101	TCP	204	80 → 49460 [PSH, A
22	3.224457261	192.168.32.100	192.168.32.101	HTTP	312	HTTP/1.1 200 OK (

Analizziamo i pacchetti che vengono scambiati:

- Pacchetti 9-11(Risoluzione DNS): Il client (192.168.32.101) chiede al server DNS (.100) che IP ha epicode.internal, il pacchetto 11 risponde con il corrispettivo indirizzo ovvero il 192.168.32.100, senza questo passaggio il browser non saprebbe a quale indirizzo inviare i dati.
- Pacchetti 13-15(Handshake TCP) Come anche nella comunicazione precedente anche qui prima di scambiare i dati HTTP viene stabilita una connessione tramite il TCP con
    1. **13[SYN]**:sincronizzazione con la porta 80(HTTP) del client
    2. **14[SYN-ACK]:** il server risponde con un ACK ovvero come se dicesse "RICEVUTO" e "SINCRONIZZIAMOCI" con il SYN da parte sua.
    3. **15[ACK]:** il client risponde con un ACK ovvero "RICEVUTO".
- Pacchetto 18(La richiesta HTTP): GET / HTTP/1.1. Il client sta chiedendo la pagina principale se aprissimo il pacchetto della richiesta tramite wireshark possiamo vedere tutto mentre su HTTPS sarebbe illeggibile.
- Pacchetto 19-21(Altre ACK di connessione e PUSH per inviare i dati dalla pagina web al client).
- Pacchetto 22(Risposta della richiesta) http/1.1 200 ok (text/html) il Server(Kali) ha accettato la richiesta (200 ok). Inoltre, qui vediamo text/html ovvero testo semplice, in HTTPS invece non vediamo il tipo di file né il contenuto

---

Analisi interna dei dati del Pacchetto 277: cliccando sul pacchetto 277 e andando in Ethernet II potremo visualizzare il Source e Destination MAC:



Se clicchiamo su Internet Protocol invece potremo andare a visualizzare l'indirizzo IP del mittente e del destinatario:

```
▾ Internet Protocol Version 4, Src: 192.168.32.101, Dst: 192.168.32.100
    0100 .... = Version: 4
```

Come possiamo vedere il mittente è il Client Windows e il destinatario il Server (Kali).

Se invece clicchiamo su Transport vedremo le porte di mittente e destinatario:

```
▾ Transmission Control Protocol, Src Port: 49460, Dst Port: 80, Seq: 1, Ack: 1, Len
    Source Port: 49460
    Destination Port: 80
```

Come possiamo vedere viene mostrata la porta randomica del mittente n 49460 e la porta di destinazione la porta 80 ovvero HTTP.

Analisi Pacchetto HTTP: Se andiamo ad analizzare il pacchetto HTTP vedremo che i dati non sono cifrati ma invece sono in chiaro:

```
▸ Frame 18: Packet, 506 bytes on wire (4048 bits), 506 bytes captured (4048 bits) o
▸ Ethernet II, Src: PCSSystemtec_92:4e:9f (08:00:27:92:4e:9f), Dst: PCSSystemtec_8a
▸ Internet Protocol Version 4, Src: 192.168.32.101, Dst: 192.168.32.100
▾ Transmission Control Protocol, Src Port: 49460, Dst Port: 80, Seq: 1, Ack: 1, Len
    Source Port: 49460
    Destination Port: 80
    [Stream index: 0]
    [Stream Packet Number: 4]
  ▸ [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 452]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 1552430459
    [Next Sequence Number: 453    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 2441284796
    0101 .... = Header Length: 20 bytes (5)

0000  08 00 27 8a 35 d2 08 00  27 92 4e 9f 08 00 45 00   ··'·5··· '·N···E·
0010  01 ec 56 7f 40 00 80 06  e0 72 c0 a8 20 65 c0 a8   ··V·@··· ·r··· e··
0020  20 64 c1 34 00 50 5c 88  35 7b 91 83 0c bc 50 18    d·4·P\· 5{····P·
0030  01 00 cd 5a 00 00 47 45  54 20 2f 20 48 54 54 50   ···Z··GE T / HTTP
0040  2f 31 2e 31 0d 0a 48 6f  73 74 3a 20 65 70 69 63   /1.1··Ho st: epic
0050  6f 64 65 2e 69 6e 74 65  72 6e 61 6c 0d 0a 43 6f   ode.inte rnal··Co
0060  6e 6e 65 63 74 69 6f 6e  3a 20 6b 65 65 70 2d 61   nnection : keep-a
0070  6c 69 76 65 0d 0a 55 70  67 72 61 64 65 2d 49 6e   live··Up grade-In
0080  73 65 63 75 72 65 2d 52  65 71 75 65 73 74 73 3a   secure-R equests:
0090  20 31 0d 0a 55 73 65 72  2d 41 67 65 6e 74 3a 20    1··User -Agent: 
00a0  4d 6f 7a 69 6c 6c 61 2f  35 2e 30 20 28 57 69 6e   Mozilla/ 5.0 (Win
00b0  64 6f 77 73 20 4e 54 20  31 30 2e 30 3b 20 57 69   dows NT  10.0; Wi
00c0  6e 36 34 3b 20 78 36 34  29 20 41 70 70 6c 65 57   n64; x64 ) AppleW
00d0  65 62 4b 69 74 2f 35 33  37 2e 33 36 20 28 4b 48   ebKit/53 7.36 (KH
00e0  54 4d 4c 2c 20 6c 69 6b  65 20 47 65 63 6b 6f 29   TML, lik e Gecko)
00f0  20 43 68 72 6f 6d 65 2f  31 32 36 2e 30 2e 30 2e    Chrome/ 126.0.0.
```

Principali Differenze Tra HTTP e HTTPS

Queste catture ci permettono di capire le principali differenze tra HTTP e HTTPS:

1. Trasparenza: Chiunque nella rete può leggere il pacchetto 18(domanda) e il pacchetto 22(risposta)
2. Efficienza vs sicurezza: L'HTTP è più veloce all'inizio perché dopo il pacchetto 15 inizia subito a inviare dati invece di attuare l'Handshake TLS per la cifratura come nel HTTPS. Ma il HTTPS nonostante renda l'avvio leggermente più lento garantisce una sicurezza infinitamente migliore.

Cosa avviene Livello per Livello: HTTP e HTTPS:

HTTP:

Livello Applicativo:

- Tutto parte dalla nostra richiesta sulla macchina **Client**(windows) digitando il dominio ***https://epicode.internal o http://epicode.internal*** nel browser. Il Browser genera un messaggio **HTTP** con il comando **GET** / per ottenere il servizio web. In **HTTP** è una semplice richiesta di testo mentre, dove dice al server "mandami questa pagina"

Livello di Presentazione:

- Questo Livello si occupa della preparazione del dato prima che venga inviato per permettere la comunicazione e anche la sicurezza in base al protocollo. In questo caso con il Protocollo **HTTP** si occupa solamente che il testo sia codificato correttamente (Es-**UTF-8**) così che il **Server**(Kali) capisca i caratteri inviati.

Livello di Sessione:

- Si occupa di gestire il dialogo tra sue macchine decidendo quando la sessione inizia, finisce e la sua gestione.

Livello di Trasporto:

- In questo livello si parla di Protocollo **TCP**. Prima di Inviare la Richiesta **GET**, il computer instaura una connessione sicura per garantire che tutto vada bene, qui come abbiamo spiegato precedentemente avviene il famoso Handshake, nel nostro caso il **client** (Windows) genera per la nostra richiesta una porta casuale (es. 49460) e punta alla porta **80**(HTTP) del **Server**(Kali)

Livello Network:

- In questo Livello si parla di **Indirizzi IP** ovvero Indirizzi logici necessari per il pacchetto per capire dove andare tra le reti. **Il protocollo IP** aggiunge un header con gli indirizzi **sorgente**(Windows) e **destinatario**(Kali).

Livello Data Link:

- In questo livello si parla di **MAC Address** necessari per sincronizzare le schede di rete tra mittente e destinatario. I dati vengono incapsulati in ciò che prende il nome in questo livello di **Frame Ethernet**, è qui che agisce il protocollo ARP per la risoluzione degli indirizzi **MAC** tramite gli **indirizzi IP**. Nel nostro caso **Windows** controlla la sua **cache ARP** per sapere a quale indirizzo **MAC** corrisponde il Server (Kali), una volta preso o trovato mette il **MAC** di **Kali** come destinazione.

Livello fisico:

In questo livello non parliamo di dati ma segnali elettrici o impulsi luminosi in base al mezzo guidante utilizzato (es-**fibra ottica, ethernet, cavo coassiale**) o non guidante (es. **wi-fi**) di questo livello fa parte la nostra **NIC** (Network Interface Card) ovvero la nostra scheda di rete. Nel nostro caso alla richiesta sulla macchina **client** (**Windows**) trasforma i bit in impulsi elettrici che viaggiano sul cavo virtuale della nostra rete di laboratorio.

Livello OSI	Nome	Elemento nel tuo Wireshark
Layer 7	Applicativo	Protocollo HTTP (GET /)
Layer 4	Trasporto	Protocollo TCP (Porta 80)
Layer 3	Rete	Protocollo IPv4 (Indirizzi IP .101 -> .100)
Layer 2	Datalink	Protocollo Ethernet (MAC Address)
Layer 1	Fisico	Bit trasmessi sulla rete del laboratorio

HTTPS:

Livello Applicativo:

- Tutto parte dalla nostra richiesta sulla macchina **Client**(windows) digitando il dominio ***https://epicode.internal o http://epicode.internal*** nel browser. Il **Browser** genera un messaggio HTTPS con il comando **GET** / per ottenere il servizio web."

Livello di Presentazione:

- Questo Livello si occupa della preparazione del dato prima che venga inviato per permettere la comunicazione e anche la sicurezza in base al protocollo. In questo caso con il Protocollo **HTTPS** si occupa della cifratura del messaggio attraverso lo scambio di chiavi avvenuto durante **l'Handshake TLS** trasformando il contenuto in qualcosa di illeggibile per chiunque altro.

Livello di Sessione:

- Si occupa di gestire il dialogo tra sue macchine decidendo quando la sessione inizia, finisce e la sua gestione.

Livello di Trasporto:

- In questo livello si parla di Protocollo **TCP**. Prima di Inviare la Richiesta **GET**, il computer instaura una connessione sicura per garantire che tutto vada bene, qui come abbiamo spiegato precedentemente avviene il famoso Handshake, nel nostro caso il **client** (Windows) genera per la nostra richiesta una porta casuale (es. 49460) e punta alla **porta 443**(HTTPS) del **server**(Kali)

Livello Network:

- In questo Livello si parla di **Indirizzi IP** ovvero Indirizzi logici necessari per il pacchetto per capire dove andare tra le reti. **Il protocollo IP** aggiunge un header con gli indirizzi **sorgente**(Windows) e **destinatario**(Kali).

Livello Data Link:

- In questo livello si parla di **MAC Address** necessari per sincronizzare le schede di rete tra mittente e destinatario. I dati vengono incapsulati in ciò che prende il nome in questo livello di **Frame Ethernet**, è qui che agisce il protocollo **ARP** per la risoluzione degli indirizzi **MAC** tramite gli **indirizzi IP.** Nel nostro caso Windows controlla la sua **cache ARP** per sapere a quale indirizzo **MAC** corrisponde il **Server** (Kali), una volta preso o trovato mette il **MAC** di **Kali** come destinazione.

Livello fisico:

In questo livello non parliamo di dati ma segnali elettrici o impulsi luminosi in base al mezzo guidante utilizzato (es-fibra ottica, ethernet, cavo coassiale) o non guidante (es. **wi-fi**) di questo livello fa parte la nostra **NIC** (Network Interface Card) ovvero la nostra scheda di rete. Nel nostro caso alla richiesta sulla macchina **client** (**Windows**) trasforma i bit in impulsi elettrici che viaggiano sul cavo virtuale della nostra rete di laboratorio.

Livello OSI	Nome	Elemento nel tuo Wireshark
Layer 7	Applicativo	Protocollo HTTP (GET /)
Layer 4	Trasporto	Protocollo TCP (Porta 443)
Layer 3	Rete	Protocollo IPv4 (Indirizzi IP .101 -> .100)
Layer 2	Datalink	Protocollo Ethernet (MAC Address)
Layer 1	Fisico	Bit trasmessi sulla rete del laboratorio